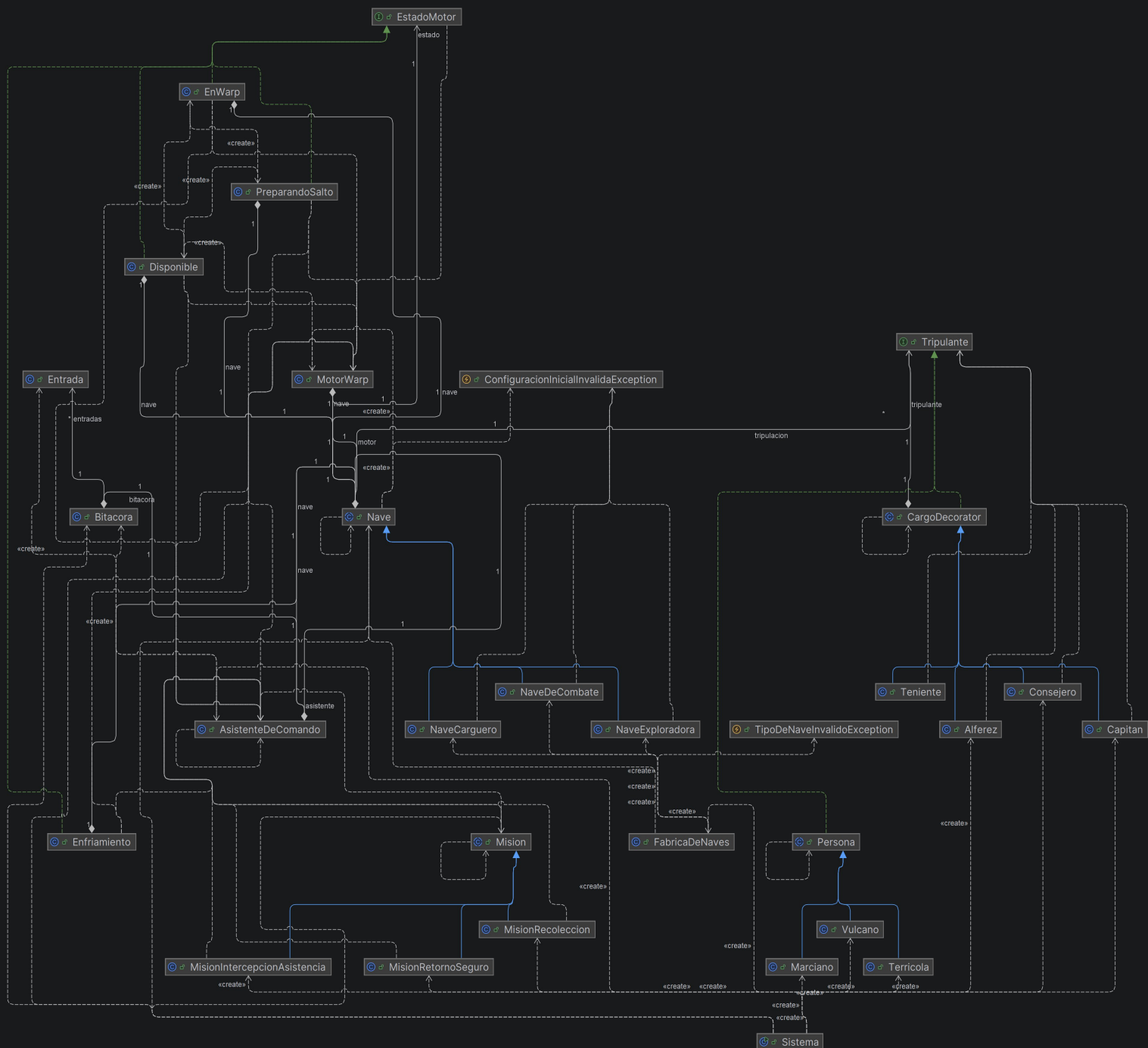


TRABAJO PRÁCTICO

GRUPAL



Grupo 05

Amelie lasillo, Juan Mc Kesey, Facundo Elizalde, Ramiro Peñalva

Introducción

Centramos el desarrollo de nuestro programa en no tener que depender de mucha implementación. Que con tal solo invocar una función de otro objeto obtener el resultado deseado.

Asistente de comando y Misiones

El asistente de comando y las misiones se realizaron simultáneamente. Para desarrollar el asistente de comando se tuvo en cuenta el principio de única responsabilidad, este es el encargado de coordinar misiones y comunicarse con la nave para verificar recursos. Es decir que si en el momento de verificar que falta combustible para la misión, este no va a llamar a la carga de combustible, solo se ocupará de informar y escribir en bitácora dicho suceso.

Las misiones siguieron estrictamente el Template Method, donde se estipula paso a paso qué debe realizar cada misión con el agregado de un Hook para aplicar las bonificaciones de energía en aquellas misiones que la otorgan. Además las misiones se comunican únicamente con el asistente de comando para verificar recursos y proseguir con el flujo de la misión.

Motor Warp y Bitácora

Estos también se realizaron simultáneamente, usando por un lado el motor warp para poder procesar los distintos estados que llevará la nave durante las misiones (disponible, preparando salto, salto warp y enfriamiento, aunque este último aún no será definido; por el momento se volverá del salto warp a estar disponible directamente). Para ello se utilizó el patrón State, con el que se pudo diseñar cada uno de sus estados a través de la implementación de una interfaz de estado del motor.

Por el otro lado, en la bitácora se registrarán todos los eventos destacados de la nave y su interacción con el universo (sean transiciones del estado del motor, recolección de recursos, errores en diversos procesos, etc.). Para ello se utiliza un ArrayList de entradas ordenadas temporalmente, cada una con una fecha, mensaje, y tipo. Esta bitácora luego será llamada por el asistente para cada registro, por ejemplo, en un fallo en un pasaje de estado en el motor.

Nave

La clase abstracta Nave se diseñó aplicando estrictamente el Encapsulamiento y el Diseño por Contratos. Es la única responsable de gestionar y proteger sus recursos internos, autolimitándose (por ejemplo, en su capacidad máxima de energía) para garantizar su integridad física sin depender de validaciones externas. Para instanciar sus distintas variantes (Exploradora, Combate y Carguero) se utilizó el patrón Factory a través de FabricaDeNaves, lo que desacopla la lógica de creación del resto del sistema y facilita la incorporación de nuevos modelos a futuro.

Estructuralmente, la nave funciona mediante una fuerte Composición: contiene a su MotorWarp (patrón State) y a su Tripulacion (patrón Decorator). Pese a integrar estos subsistemas complejos, respeta el principio de Única Responsabilidad: se limita a administrar sus recursos y exponer su estado operativo, delegando todo el control de flujo de las misiones y los registros en bitácora a su Asistente de Comando. Esto garantiza un código de bajo acoplamiento y alta cohesión.

Tripulación

El módulo de Tripulacion fue diseñado aplicando como eje central el patrón estructural Decorator. Esto permite asignar y combinar dinámicamente diferentes cargos y responsabilidades (como Capitan o Alférez) a un tripulante base en tiempo de ejecución, evitando una jerarquía rígida y una explosión combinatoria de subclases por herencia.

Gracias a este diseño, operaciones complejas como el cálculo de nómina (calcularHaber) se resuelven de manera elegante mediante Polimorfismo. La nave interactúa únicamente con la interfaz Tripulante, y cada decorador apilado se encarga de sumar su respectiva bonificación al sueldo base de forma recursiva y transparente.

Esta arquitectura cumple estrictamente con el principio Open/Closed (Abierto a la extensión, Cerrado a la modificación). El sistema está preparado para incorporar nuevos rangos, roles o razas a la flota en el futuro sin necesidad de alterar el código de las clases base existentes, asegurando un alto nivel de escalabilidad y mantenimiento.

Conclusión

A modo de conclusión, el desarrollo de este simulador demuestra la aplicación exitosa y articulada de conceptos avanzados de la Programación Orientada a Objetos. La integración estratégica de patrones de diseño (State, Decorator, Factory, Template Method), junto con el cumplimiento de los principios SOLID, permitió construir un sistema altamente modular, cohesivo y con bajo acoplamiento.

Además, el uso riguroso del Diseño por Contratos y el manejo centralizado de excepciones de dominio garantizan la robustez del software. El sistema es capaz de gestionar escenarios complejos y fallos en tiempo de ejecución de manera atómica, abortando operaciones inválidas sin corromper el estado interno de los objetos. En definitiva, el proyecto resulta en una arquitectura segura, escalable y fácil de mantener, reflejando estándares de calidad propios del desarrollo de software profesional.